

CSc 453 — Assignment 8

Christian Collberg
University of Arizona, Department of Computer Science

April 21, 2026

Your task is to write a code generator and code optimizer for a small subset of Luca. You will be given a front-end that generated quadruple code from Luca programs. For this assignment you can code in the language of your choice.

Important notes:

- You don't have to implement procedures.
- You don't have to implement array bounds checks.
- You don't have to implement real and char and string types.
- You can write your code generator in any reasonable language (C, C++, Python, Java, ...).
- You can get an extra 50 points by implementing the extensions.

1 Getting Started

This assignment can be done in teams of one or two.

To get started, follow these instructions:

1. Unzip `assignment-8.zip` which gives you these files:

```
assignment-8/assignment-8.pdf
assignment-8/firstname_lastname/tests/*.luc
assignment-8/firstname_lastname/quad_files/*.quad
assignment-8/lucac_macos_m1
assignment-8/lucac_linux_x86_64
assignment-8/firstname_lastname/collaboration.json
assignment-8/firstname_lastname/survey.json
```

2. Then, assuming your name is Pat Jones, execute these commands:

```
> cd assignment-8
> mv firstname_lastname pat_jones
```

If you are working in a group of two, pick one of your names.

3. Implement your code generator in `firstname_lastname`.

2 Implementation

Your task is to write a code generator and optimizer for the language LUCA.

1. The interpreter should be named `luca_mips`.
2. You first compile your program to quadruples code using the provided Luca front-ends:

```
> lucac_macos_m1      -i x.luc -q x.quad
> lucac_linux_x86_64 -i x.luc -q x.quad
```

And then you run your code generator on that intermediate code, producing a Mips assembly code file `x.s` as output:

```
> luca_mips x.quad x.s
```

And then you can run the generated code using the `Spim` Mips emulator:

```
> spim -f x.s
```

3. To turn on the optimizer, add the `-O` option as the third argument:

```
> luca_mips x.vm x.mips -O
```
4. **This assignment can be coded in any “reasonable” language of your choice: C, C++, Java, Python,**
5. Your code generator should be able to handle arbitrarily large programs. I.e., it should generate correct code for deeply nested control structures, very large expressions, etc.

2.1 Language subset

While the front ends you have been given support the full Luca language, **your code generator only has to support a small subset:**

- The types `REAL`, `STRING`, and `CHAR` are not supported.
- Procedures and procedure calls are not supported.
- You don’t have to check for division by zero or array index out of bounds.

3 Quad File

The quad file produced by the front-end is a text file with one quad per line. The format is

```
OPERATOR attribute=value attribute=value attribute=value attribute=value ...
```

For example, a binary expression quad looks like this, where `resultId`, `leftId`, `rightId` are variable references:

```
BinExpr op=+ typeId=1 resultId=23 leftId=20 rightId=22 pos=8
```

As an example, consider this program:

```
PROGRAM P;  
  
VAR I : INTEGER;  
VAR J : INTEGER;  
BEGIN  
    I := 0;  
    J := 0;  
    WRITE J+I; WRITELN;  
END.
```

This is the corresponding Quad file:

```
TypeDecl symbol=INTEGER id=1 size=4 pos=0  
TypeDecl symbol=CHAR id=3 size=1 pos=0  
TypeDecl symbol=REAL id=2 size=4 pos=0  
TypeDecl symbol=STRING id=4 size=0 pos=0  
TypeDecl symbol=$ADDRESS id=7 size=4 pos=0  
TypeDecl symbol=BOOLEAN id=8 size=4 pos=0  
VarDecl symbol=I id=13 kind=global typeId=1 offset=0 pos=3  
VarDecl symbol=J id=14 kind=global typeId=1 offset=4 pos=4  
VarDecl symbol=$0 id=15 kind=temp typeId=7 offset=-1 pos=0  
VarDecl symbol=$1 id=16 kind=temp typeId=1 offset=-1 pos=0  
VarDecl symbol=$2 id=17 kind=temp typeId=7 offset=-1 pos=0  
VarDecl symbol=$3 id=18 kind=temp typeId=1 offset=-1 pos=0  
VarDecl symbol=$4 id=19 kind=temp typeId=7 offset=-1 pos=0  
VarDecl symbol=$5 id=20 kind=temp typeId=7 offset=-1 pos=0  
VarDecl symbol=$6 id=21 kind=temp typeId=7 offset=-1 pos=0  
VarDecl symbol=$7 id=22 kind=temp typeId=7 offset=-1 pos=0  
VarDecl symbol=$8 id=23 kind=temp typeId=7 offset=-1 pos=0  
Version major=3 minor=0 pos=1  
ProcBegin symbol=$MAIN id=0 formalCount=0 formalSize=0 localCount=2 localSize=8 pos=1  
AddressOf symbol=I id=13 typeId=7 resultId=15 pos=6  
LoadLit value=0 typeId=1 resultId=16 pos=6  
Store typeId=1 leftId=15 rightId=16 pos=6  
AddressOf symbol=J id=14 typeId=7 resultId=17 pos=7  
LoadLit value=0 typeId=1 resultId=18 pos=7  
Store typeId=1 leftId=17 rightId=18 pos=7  
AddressOf symbol=J id=14 typeId=7 resultId=19 pos=8  
Load typeId=1 resultId=20 desId=19 pos=8  
AddressOf symbol=I id=13 typeId=7 resultId=21 pos=8  
Load typeId=1 resultId=22 desId=21 pos=8  
BinExpr op=+ typeId=1 resultId=23 leftId=20 rightId=22 pos=8  
Write typeId=1 exprId=23 pos=8  
WriteLn pos=8  
ProcEnd symbol=$MAIN id=0 pos=1
```

3.1 Part 1: Standard Type Declarations

Standard types are declared first in the file. They are referenced by their `Id`. The integer type is symbol number 1, for example:

```
TypeDecl symbol=INTEGER id=1 size=4 pos=0
TypeDecl symbol=CHAR id=3 size=1 pos=0
TypeDecl symbol=REAL id=2 size=4 pos=0
TypeDecl symbol=STRING id=4 size=0 pos=0
TypeDecl symbol=$ADDRESS id=7 size=4 pos=0
TypeDecl symbol=BOOLEAN id=8 size=4 pos=0
```

3.2 Part 2: Declarations

Next follows declarations of variables and types. You can think of this as the symbol table embedded as Quad instructions:

```
VarDecl symbol=I id=13 kind=global typeId=1 offset=0 pos=3
VarDecl symbol=J id=14 kind=global typeId=1 offset=4 pos=4
VarDecl symbol=$0 id=15 kind=temp typeId=7 offset=-1 pos=0
VarDecl symbol=$1 id=16 kind=temp typeId=1 offset=-1 pos=0
VarDecl symbol=$2 id=17 kind=temp typeId=7 offset=-1 pos=0
VarDecl symbol=$3 id=18 kind=temp typeId=1 offset=-1 pos=0
VarDecl symbol=$4 id=19 kind=temp typeId=7 offset=-1 pos=0
VarDecl symbol=$5 id=20 kind=temp typeId=7 offset=-1 pos=0
VarDecl symbol=$6 id=21 kind=temp typeId=7 offset=-1 pos=0
VarDecl symbol=$7 id=22 kind=temp typeId=7 offset=-1 pos=0
VarDecl symbol=$8 id=23 kind=temp typeId=7 offset=-1 pos=0
```

The attribute ‘‘kind’’ can be one of `global`, `formal`, `local`, or `temp`. Temporary variables are named `$<number>`. Variables are referenced by their Id number.

3.3 Part 3: The Main Program

The final part is the main program, which begins with `ProcBegin symbol=$MAIN` and ends with `ProcEnd`:

```
Version major=3 minor=0 pos=1
ProcBegin symbol=$MAIN id=0 formalCount=0 formalSize=0 localCount=2 localSize=8 pos=1
AddressOf symbol=I id=13 typeId=7 resultId=15 pos=6
LoadLit value=0 typeId=1 resultId=16 pos=6
Store typeId=1 leftId=15 rightId=16 pos=6
AddressOf symbol=J id=14 typeId=7 resultId=17 pos=7
LoadLit value=0 typeId=1 resultId=18 pos=7
Store typeId=1 leftId=17 rightId=18 pos=7
AddressOf symbol=J id=14 typeId=7 resultId=19 pos=8
Load typeId=1 resultId=20 desId=19 pos=8
AddressOf symbol=I id=13 typeId=7 resultId=21 pos=8
Load typeId=1 resultId=22 desId=21 pos=8
BinExpr op=+ typeId=1 resultId=23 leftId=20 rightId=22 pos=8
Write typeId=1 exprId=23 pos=8
WriteLn pos=8
ProcEnd symbol=$MAIN id=0 pos=1
```

3.4 Arithmetic

Let’s look in some more detail at the statement `WRITE J+I;`. I’ve added comments to the instructions to explain what they do:

```

# Load the address of global variable J into the temporary with Id=19 (named $4):
AddressOf symbol=J id=14 typeId=7 resultId=19 pos=8
# Load the value of global variable J into temporary 20 from
# the address stored in temporary 19:
Load typeId=1 resultId=20 desId=19 pos=8

# Load I into temporary 22:
AddressOf symbol=I id=13 typeId=7 resultId=21 pos=8
Load typeId=1 resultId=22 desId=21 pos=8

# Compute I+J into temporary 23:
BinExpr op=+ typeId=1 resultId=23 leftId=20 rightId=22 pos=8

# Print the value in temporary 23:
Write typeId=1 exprId=23 pos=8

```

3.5 Arrays

Consider this program:

```

PROGRAM P;

TYPE A = ARRAY 100 OF INTEGER;

VAR x : A;
VAR i : INTEGER;
BEGIN
    i := 23;
    x[i] := 45;
    WRITE x[i] - 45; WRITELN;
END.

```

The array A is symbol 13, and we can see that x is declared to be of this type:

```

ArrayDecl symbol=A id=13 size=400 elementTypeId=1 elementCount=100 pos=3
VarDecl symbol=x id=14 kind=global typeId=13 offset=0 pos=5
VarDecl symbol=i id=15 kind=global typeId=1 offset=400 pos=6

```

Here's the code for the statement `x[i] := 45;`:

```

# Load the value of i into temporary 19:
AddressOf symbol=i id=15 typeId=7 resultId=18 pos=9
Load typeId=1 resultId=19 desId=18 pos=9

# Load the address of array x into temporary 20:
AddressOf symbol=x id=14 typeId=7 resultId=20 pos=9

# Compute the address of array element x[i] into temporary 21.
# In other words, compute
# temp21 = temp20 + temp18*elementSize
# Get elementSize by following the links from
# arrayId=13 to
# elementTypeId=1, to
# TypeDecl symbol=INTEGER id=1 size=4 pos=0"

```

```

IndexOf typeId=7    arrayId=13    resultId=21    baseId=20    indexId=19    pos=9

# Load literal integer 45 into temporary 22:
LoadLit value=45    typeId=1    resultId=22    pos=9

# Store 45 into array element x[i]:
Store typeId=1    leftId=21    rightId=22    pos=9

```

3.6 Records

Consider this program:

```

PROGRAM P;

TYPE R = RECORD [a: INTEGER; b : INTEGER];

VAR x : R;
VAR i : INTEGER;
BEGIN
  x.b := 99;
  x.a := 256724346;
  WRITE x.b; WRITELN;
END.

```

The array A is symbol 13, and we can see that x is declared to be of this type:

```

# Declare the record R:
RecordDecl symbol=R id=13 size=8 pos=3

# Declare the two fields of R. The most important part here
# is the byte offset with the record:
FieldDecl symbol=a id=14 recordId=13 typeId=1 offset=0 pos=3
FieldDecl symbol=b id=15 recordId=13 typeId=1 offset=4 pos=3

# Declare the record variable x:
VarDecl symbol=x id=16 kind=global typeId=13 offset=0 pos=5

```

And now for the statement `x.b := 99;`:

```

# Load the address of x into temp18:
AddressOf symbol=x id=16 typeId=7 resultId=18 pos=8

# Compute the address of field b within R. In other words, compute the address
# temp19 = temp18 + 4
# where 4 is the offset of field b which we get from the FieldDecl instruction:
FieldOf field=b record=R fieldId=15 recordId=13 typeId=7 resultId=19 baseId=18 pos=8

# Load literal 99 into temp20 and store into x.b:
LoadLit value=99 typeId=1 resultId=20 pos=8
Store typeId=1 leftId=19 rightId=20 pos=8

```

3.7 Control Structures

Consider this program:

```
PROGRAM P;

VAR i : INTEGER;
BEGIN
  i := 10;
  IF i = 10 THEN
    WRITE 0;
  ELSE
    WRITE 99;
  ENDIF;
  WRITELN;
END.
```

Here's the generated Quad code, with explanations:

```
# temp17 = value of i
AddressOf      symbol=i      id=13  typeId=7      resultId=16  pos=6
Load  typeId=1  resultId=17  desId=16  pos=6

# temp18 = 10
LoadLit value=10      typeId=1      resultId=18  pos=6

# If temp17 = temp18 goto Label23
Branch type=INTEGER  op==  label=23      leftId=17      rightId=18  pos=6

# Else goto Label24
Goto  label=24      pos=6

# Label23 is here
Label  label=23      pos=6

# Write 0
LoadLit value=0 typeId=1      resultId=19  pos=7
Write  typeId=1      exprId=19  pos=7

# Goto Label15
Goto  label=15      pos=6

# Label 24 is here
Label  label=24      pos=6

# Write 99
LoadLit value=99      typeId=1      resultId=20  pos=9
Write  typeId=1      exprId=20  pos=9

# Label15 is here
Label  label=15      pos=11
```

3.8 Procedures [Extension only!]

Consider this program:

```
PROGRAM P;

VAR x : INTEGER;
VAR a : INTEGER;

PROCEDURE Q (
  a : INTEGER;
  b : CHAR;
  c : REAL);
BEGIN
  x := a + TRUNC(c);
END;

BEGIN
  x := 23; a := 34;
  Q(a+1, '0', 5.14);
  WRITE x - 40; WRITELN;
END.
```

Here's the generated Quad code for the procedure Q:

```
# Declare formal parameters:
FormalDecl symbol=a id=16 typeId=1 offset=0 mode=VAL formalNumber=1 pos=7
FormalDecl symbol=b id=17 typeId=3 offset=4 mode=VAL formalNumber=2 pos=8
FormalDecl symbol=c id=18 typeId=2 offset=8 mode=VAL formalNumber=3 pos=9

# Declare procedure Q:
ProcBegin symbol=Q id=15 formalCount=3 formalSize=12 localCount=0 \
  localSize=0 formalIds=17,18,16 pos=6
...
ProcEnd symbol=Q id=15 pos=6
```

And here's the code generated for the call Q(a+1, '0', 5.14):

```
# First argument a+1:
AddressOf symbol=a id=14 typeId=7 resultId=30 pos=16
Load typeId=1 resultId=31 desId=30 pos=16
LoadLit value=1 typeId=1 resultId=32 pos=16
BinExpr op=+ typeId=1 resultId=33 leftId=31 rightId=32 pos=16
Actual exprId=33 formalId=16 pos=16

# Second argument '0'
LoadLit value=0 typeId=3 resultId=34 pos=16
Actual exprId=34 formalId=17 pos=16

# Third argument 5.14:
LoadLit value=5.14 typeId=2 resultId=35 pos=16
Actual exprId=35 formalId=18 pos=16

# The call to Q:
ProcCall symbol=Q id=15 pos=16
```

4 Code Optimization

Extend the Mips code generator with a simple “peephole optimizer”. It does the following:

1. Examine a “window” of instructions, maybe 3-4 instructions long.
2. Improve the code in the window.
3. Slide window down one instruction
4. Repeat until no more changes can be made.

Note:

- You can implement this transformation either over the Quad instructions or (better) over the generated Mips code.
- Make sure that you add the `-O` option to turn optimization on and off.

Implement at least these optimizing transformations:

4.1 Algebraic simplification

These are some possibilities:

$x \leftarrow x + 0;$	$\Rightarrow$	
$x \leftarrow x - 0;$	$\Rightarrow$	
$x \leftarrow x * 1;$	$\Rightarrow$	
$x \leftarrow 1 * 1;$	$\Rightarrow$	$x \leftarrow 1$
$x \leftarrow x / 1;$	$\Rightarrow$	
$f \leftarrow f / 2.0;$	$\Rightarrow$	$f \leftarrow f * 0.5;$

4.2 Reduction in strength

Multiplications (and divisions) by constants can be replaced by cheaper sequences of shifts and adds:

$x \leftarrow x * 32 \Rightarrow x \leftarrow \text{SHL}(x, 5);$

$x \leftarrow x * 100$
 $\Downarrow$
 $x \leftarrow x * (64 + 32 + 4)$
 $\Downarrow$
 $x \leftarrow x * 64 + x * 32 + x * 4$
 $\Downarrow$
 $x \leftarrow \text{SHL}(x, 6) + \text{SHL}(x, 5) + \text{SHL}(x, 2)$

4.3 Jump-to-jump elimination

Complicated boolean expressions (with many **and**, **or**, **nots**) can easily produce lots of jumps to jumps which can be eliminated:

	if a < b goto L1		if a < b goto L3
	...		...
L1:	goto L2	$\Rightarrow$	L1: goto L3
	...		...
L2:	goto L3		L2: goto L3

4.4 Constant Folding

Replace constant subexpressions by their value.

5 Academic Integrity

- Don't show your code to anyone outside your team.
- Don't read anyone else's code.
- Don't discuss the details of your code with anyone.
- If you need help with the assignment see the TAs or the instructor.

5.1 Use of AI

You must write all code by hand without the use of any AI tools.

Q: Can I upload the assignment to an AI tool to explain it to me?

A: No. If you don't understand the assignment, ask the TAs.

Q: Can I use an AI tool to generate test cases?

A: No.

Q: In fact, are there *any* possible use cases for which it is OK for me to use AI tools in this assignment?

A: No, there are *no* such use cases.

Q: I will be using AI coding tools in my future job. So why can't I prepare for that by using such tools in this class?

A: This is a compiler class. The goal is to learn about compilers. If you want to learn about AI, take an AI class.

Q: But you will never be able to tell if my code was generated by an AI coding tool!

A: ROTFL!

6 Submission

To submit, follow these instructions:

1. Verify that your submission contains a working **Makefile** and *all* the files necessary to build the code generator. Your program must compile out of the box. The graders will *not* try to debug your program or your makefile for you.
2. You should test the code generator on *lectura*.
3. Fill out `collaboration.json` with information about your group.

4. Optionally fill out `survey.json` to give me feedback about the assignment.
5. Package up your answers and verify that they look OK:

```
> zip -r pat_jones.zip pat_jones
> assignment-8/validator.sh pat_jones.zip
==== This is the LigerLabs assignment validator ====
...
==== 6: Checking that all problems have been answered.
==== 7: Checking that all answer files have valid structure.
```

6. Upload `pat_jones.zip` to d2l. If you are working in a group of two, you should only submit once.

7 Grading

38 test cases have been provided for you in `assignment-8/tests/*/*.luc`. The corresponding intermediate code files are in `quad_files/*/*.vm`.

Each test case will give you 0 points if you get it wrong, 1.5 point if you get it right.

Each optimization you implement will give you 8 points, for a total of 32 points.

You can get an additional 11 points from secret test cases.

You can get an additional 50 points if you implement the extensions in Section 8. These can be used to offset loss of points in the other assignments.

8 Graded Extensions

For an extra 50 points (for a total of 150 for this assignment), you can implement the rest of the Luca language. Your Mips code generator should support

- recursive procedures [30 points],
- `REAL` arithmetic and `STRING` and `CHAR` types [15 points],
- issuing an error message when an array index is out of range [5 points].